Contenido

Introducción 3

Instalación 4

- Preparación del entorno 4
- Construcción del docker 4
- Comprobación de su funcionamiento 4

Reconocimiento 6

- Nmap 6
- TLS y autenticación 6
- Mongo-db sin control de acceso 7

Ataques 8

- NoSQL injection 8
- Lectura y escritura anónima en MongoDB 8
- Elasticsearch abierto 8

Mejora de la defensa 10

- Bloquear NoSQL Injection 10
- Autenticación en MongoDB 10
- Ocultar Elasticsearch 11
- Mejorar paginación 11
- TLS 12
- Ratelimit 12
- Intentos de login fallidos 12
- Resto de medidas 13
  - Backups 13
  - Picos de búsqueda 13

Verificación de la defensa 14

- NoSQL Injection y rate limit 14
- Autenticación en MongoDB 15
- Acceso a Elasticsearch 15

Conclusión 16

Bibliografía 17

Ilustraciones

Ilustración 1. Contenedores en funcionamiento (05/11/2026) 4
Ilustración 2. Creación de artículos con script (05/11/2026) 4
Ilustración 3. Acceso a FastAPI (05/11/2026) 4
Ilustración 4. Acceso a artículos (05/11/2026) 5
Ilustración 5. Autenticación en FastAPI (05/11/2026) 5
Ilustración 6. Acceso a pedidos (05/11/2026) 5
Ilustración 7. Descubrimiento de los contenedores a través de Nmap (05/11/2026) 6
Ilustración 8. Error al intentar acceder a FastAPI con HTTPS (05/11/2026) 6
Ilustración 9. Error al intentar acceder a Elasticsearch con HTTPS (05/11/2026) 6
Ilustración 10. Acceso a los datos de Elasticsearch con un comando curl (05/11/2026) 6
Ilustración 11. Acceso a los índices de Elasticsearch con un comando curl 6
Ilustración 12. Advertencia sobre falta de control de acceso al acceder a MongoDB (05/11/2026) 7
Ilustración 13. Ejecución de comandos en MongoDB sin autenticación (05/11/2026) 7
Ilustración 14. Acceso legítimo a FastAPI con credenciales de usuario (05/11/2026) 8
Ilustración 15. Ataque NoSQL vulnerando el parámetro de la contraseña (05/11/2026) 8
Ilustración 16. Inserción de un usuario en la base de datos sin autenticación (05/11/2026) 8
Ilustración 17. Búsqueda y lectura de todos los artículos (05/11/2026) 9
Ilustración 18. Borrado del índice exitoso a través de un comando curl (05/11/2026) 9
Ilustración 19. Cambio de la función login para evitar ataques NoSQLi (06/11/2026) 10
Ilustración 20. Creación de un usuario administrador para MongoDB (06/11/2026) 10
Ilustración 21. Indicación en el YAML de que MongoDB requiere autenticación (06/11/2026) 10
Ilustración 22. Escritura de las credenciales de MongoDB en la configuración de FastAPI (06/11/2026) 10
Ilustración 23. Comentamos la instrucción del puerto de Elasticsearch para desactivarla (06/11/2026) 11
Ilustración 24. Configuración del script de inicialización para apuntar a Elasticsearch (06/11/2026) 11
Ilustración 25. Importación de la libreria quote_plus para habilitar la paginación (06/11/2026) 11
Ilustración 26. Uso de la librería quote_plus en la función para obtener artículos (06/11/2026) 11
Ilustración 27. Adición de la biblioteca slowapi en requirements.txt (06/11/2026) 12
Ilustración 28. Inicialización y uso del limitador en el método login del YAML (06/11/2026) ... 12
Ilustración 29. Configuración de alertas de intentos fallidos a través de la biblioteca logging en el YAML (06/11/2026) 13
Ilustración 30. Reconstrucción del entorno (06/11/2026) 14
Ilustración 31. Ejecutamos el script de inicialización dentro de Elasticsearch (06/11/2026) 14
Ilustración 32. Intento de acceso a FastAPI con NoSQLi y credenciales no válidas (06/11/2026) 14
Ilustración 33. Lectura de los errores dentro del log de FastAPI (06/11/2026) 14
Ilustración 34. Secuencia de intentos de autenticación no válidos hasta el bloqueo (06/11/2026) 14

Ilustración 35. Lectura de los intentos fallidos y del bloqueo en el log de FastAPI (06/11/2026) 15
Ilustración 36. Error al intentar leer en MongoDB (06/11/2026) 15
Ilustración 37. Intento fallido de lectura de los índices de Elasticsearch (06/11/2026) 15
Ilustración 38. Acceso a Elasticsearch a través de su IP 15

Introducción

En esta actividad, vamos a montar tres contenedores que contendrán una FastAPI, que se conectará con MongoDB y Elasticsearch. Partiendo de los códigos facilitados, se analizarán sus posibles vulnerabilidades mediante la ejecución de ataques controlados sobre el entorno de laboratorio. Posteriormente, aplicaremos diversas medidas de seguridad para endurecer la configuración de los servicios y lanzaremos los mismos ataques previos con el fin de comprobar si siguen siendo efectivos. Todo el ejercicio se llevará a cabo en una máquina virtual Kali Linux.

Este tipo de prácticas permiten demostrar los riesgos asociados a configuraciones inseguras y a la confianza implícita entre sistemas. Adquieren especial relevancia los principios de Zero Trust y mínimo privilegio que, como punto principal, señalan la necesidad de desconfiar de cualquier dispositivo, persona o servidor que intenten acceder a un contenido, y garantizar que cada usuario solo tiene los privilegios mínimos necesarios para realizar su trabajo. El Reglamento General de Protección de Datos (RGPD) establece la necesidad de implantar las medidas suficientes y necesarias de seguridad para garantizar la seguridad de los datos, especialmente cuando se trata de datos sensibles.

En la conclusión, analizaremos los resultados obtenidos, tanto en la explotación de vulnerabilidades como en su mitigación, y cómo se relacionan con este tipo de principios y reglamentos.

Instalación

Preparación del entorno

Hemos visto que en la práctica se piden artículos y pedidos, los cuales no están creados. Usaremos como base los códigos que aparecen en el enunciado de la actividad. Vamos a modificar el main.py que se nos da en el enunciado para añadir una sección de pedidos (con control de errores para validar errores en los datos introducidos). Estos cambios se pueden ver en el main_old.py.

Crearemos un init_mongo.js también, donde especificamos los usuarios y pedidos. En el repositorio, se encuentra como init_mongo_old.js. Por último, escribimos un script init_elastic.sh para añadir los artículos, que en el repositorio se encuentra como init_elastic_old.sh.

Construcción del docker

Vamos a utilizar Docker-compose.yml para levantar los servicios con *Docker compose up -d –build*, y quedarían los contenedores corriendo así:

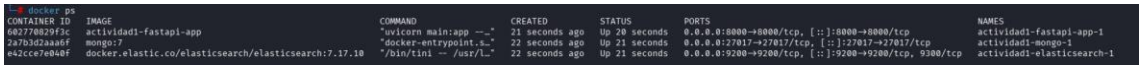


Ilustración 1. Contenedores en funcionamiento (05/11/2026)

Lanzamos el script para crear artículos



Ilustración 2. Creación de artículos con script (05/11/2026)

Comprobación de su funcionamiento

FastAPI funciona

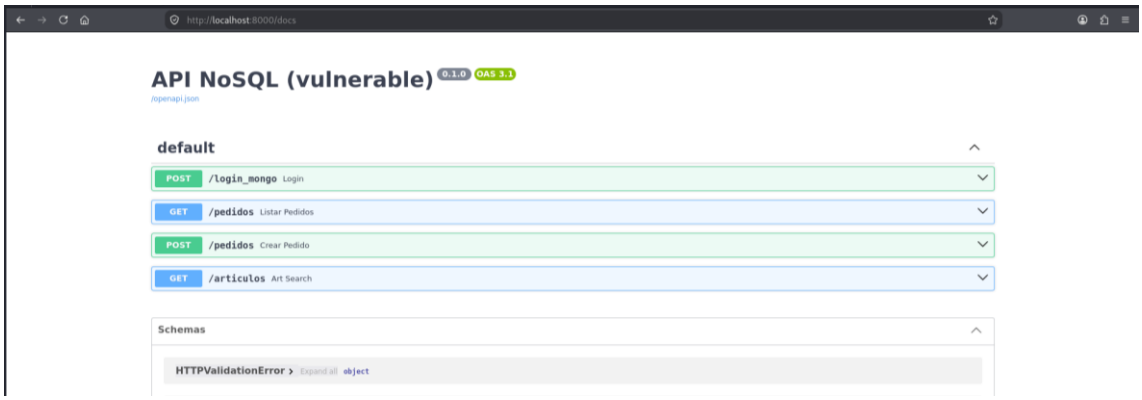


Ilustración 3. Acceso a FastAPI (05/11/2026)

Artículos también funciona

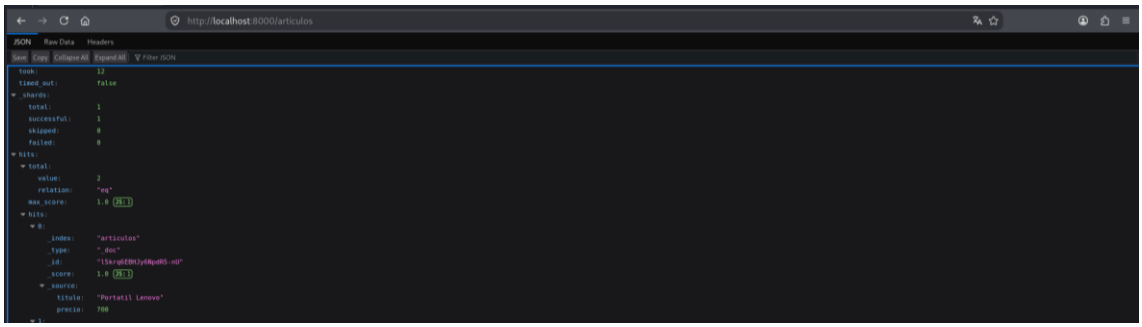


Ilustración 4. Acceso a artículos (05/11/2026)

Los usuarios de FastAPI están correctos.



Ilustración 5. Autenticación en FastAPI (05/11/2026)

Podemos visualizar correctamente también los pedidos

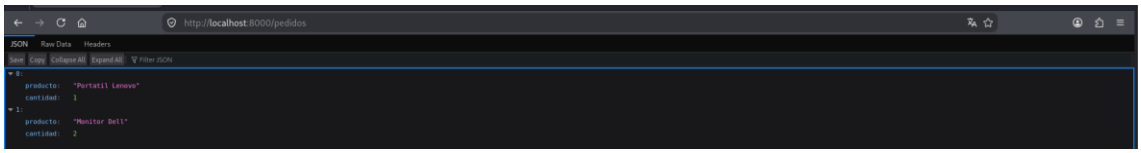


Ilustración 6. Acceso a pedidos (05/11/2026)

Reconocimiento

Nmap

Podemos encontrar los 3 servicios con un escaneo de nmap



Ilustración 7. Descubrimiento de los contenedores a través de Nmap (05/11/2026)

En este escaneo ya podemos descubrir una vulnerabilidad en Elasticsearch: cualquiera puede ejecutar el método "DELETE" (y borrar información).

TLS y autenticación

Si tratamos de hacer una conexión HTTPS a Elasticsearch y a FastAPI, veremos que nos da error, concluyendo que no tiene TLS.



Ilustración 8. Error al intentar acceder a FastAPI con HTTPS (05/11/2026)



Ilustración 9. Error al intentar acceder a Elasticsearch con HTTPS (05/11/2026)

Si hacemos una conexión HTTP, nos da los datos del cluster (nombre y versión) y versión de Lucene sin autenticarnos. También puedo ver los índices.

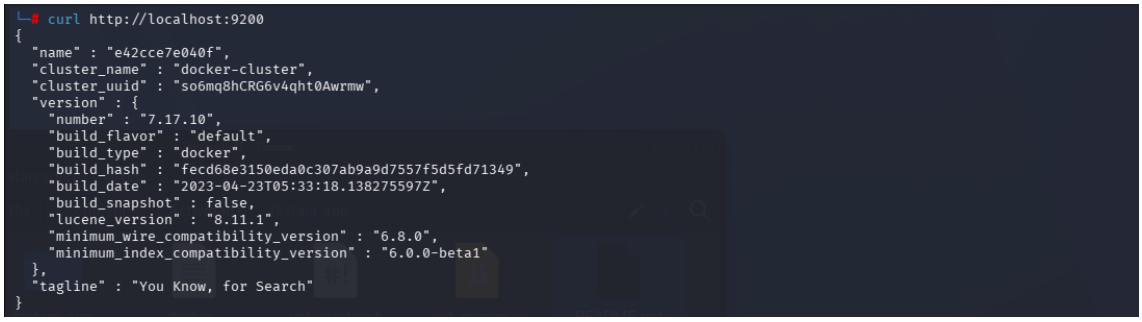


Ilustración 10. Acceso a los datos de Elasticsearch con un comando curl (05/11/2026)

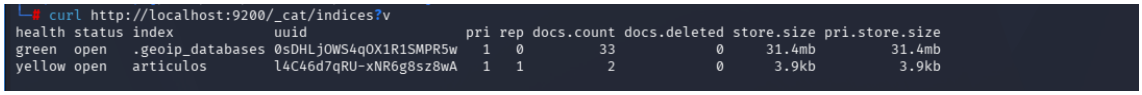


Ilustración 11. Acceso a los índices de Elasticsearch con un comando curl

Mongo-db sin control de acceso

Cuando accedemos a Mongo, encontramos una advertencia sobre la falta de control de acceso.

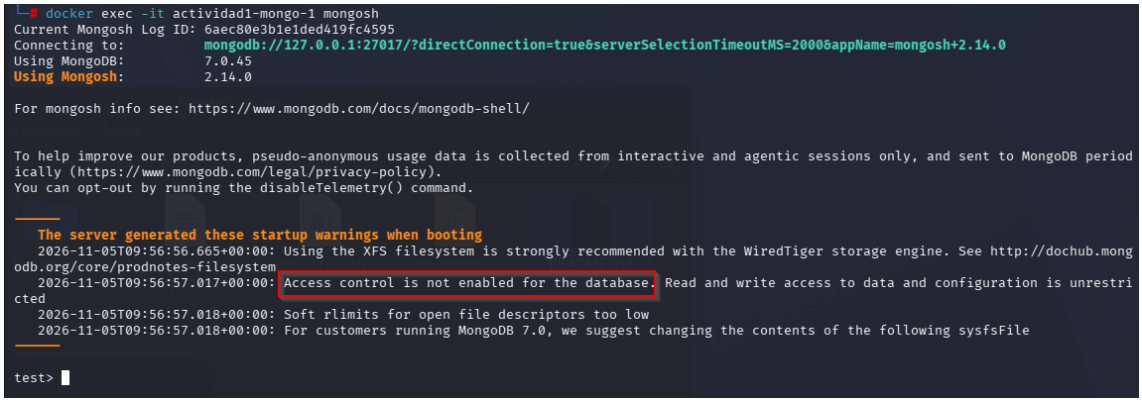


Ilustración 12. Advertencia sobre falta de control de acceso al acceder a MongoDB (05/11/2026)

Podemos ejecutar comandos libremente.

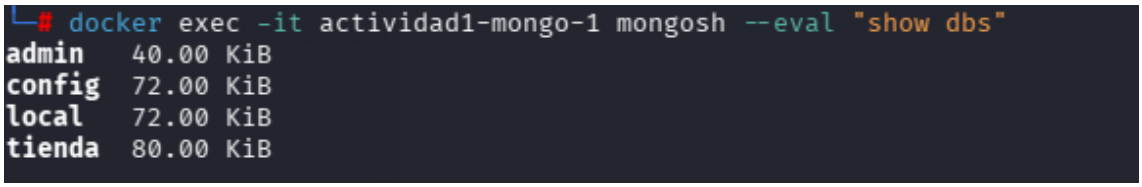


Ilustración 13. Ejecución de comandos en MongoDB sin autenticación (05/11/2026)

Todo lo anterior nos hace encontrarnos con una superficie de ataque bastante amplia, por la exposición de los servicios sin medidas de autenticación o protección.

Ataques

NoSQL injection

Tenemos un login legítimos con los usuarios de FastAPI

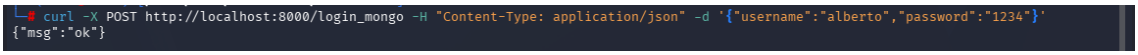


Ilustración 14. Acceso legítimo a FastAPI con credenciales de usuario (05/11/2026)

Si modificamos el password y ponemos {"$ne":"x"}, podremos obtener acceso igualemente. Esto se traduce en decirle al validador que compruebe que el password del usuario es distinto de "x", lo cual siempre funcionará mientras esa no sea la contraseña.



Ilustración 15. Ataque NoSQL vulnerando el parámetro de la contraseña (05/11/2026)

Lo acepta.

Lectura y escritura anónima en MongoDB

Entrando en MongoDB, podemos comprobar que no nos pide autenticación, tal y como vimos en el apartado anterior y que podemos ejecutar comandos de lectura, como poder ver las bases de datos. Esto nos va a permitir leer cualquier información sin autenticación.

Además, también podemos escribir una entrada nueva también.



Ilustración 16. Inserción de un usuario en la base de datos sin autenticación (05/11/2026)

Elasticsearch abierto

Ya vimos que podíamos acceder al índice en el apartado anterior. Tenemos acceso libre para buscar, por ejemplo, artículos.

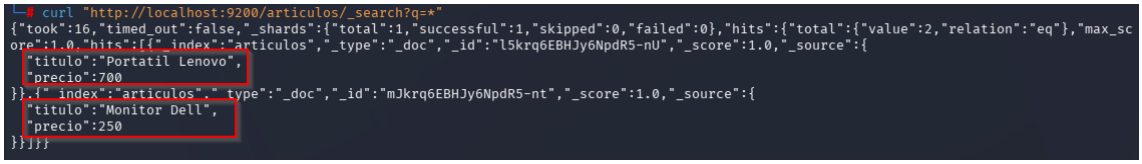


Ilustración 17. Búsqueda y lectura de todos los artículos (05/11/2026)

Y también es posible ejecutar, por ejemplo, un borrado del índice exitosamente.

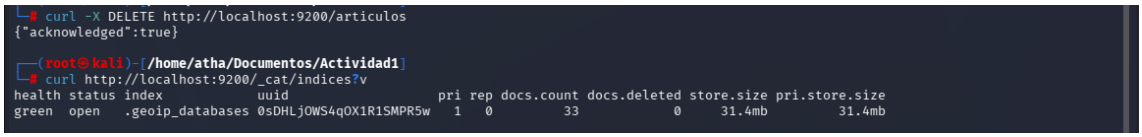


Ilustración 18. Borrado del índice exitoso a través de un comando curl (05/11/2026)

Ha desaparecido "artículos" del índice. Borrado correcto.

Mejora de la defensa

Bloquear NoSQL Injection

Para evitar esa concatenación que acaba formando una expresión regular, haremos que data (los datos introducidos) sean un objeto tipo Login (username + password) definido previamente. De esta manera, tenemos data.username por un lado y data.password por otro.

```python
async def login(data: Login):

    # ❌ NoSQLi: acepta operadores $ en credenciales
    user = db.usuarios.find_one({"username": data.username,
                                 "password": data.password}, {"_id":0})
    if user: return {"msg":"ok"}
    raise HTTPException(status_code=401, detail="Credenciales incorrectas")
```

Ilustración 19. Cambio de la función login para evitar ataques NoSQLi (06/11/2026)

Autenticación en MongoDB

Tenemos que crear un usuario administrador, que lo haremos en el init_mongo.js. Usamos el método createUser porque vamos a crear un usuario para la bbdd. Los insertMany que ya tenemos sirven para la colección de la aplicación FastAPI, los que usamos para autenticarnos allí.

```
db.createUser({
    user: "admin",
    pwd: "Iamtheadmin",
    roles: [
        {
            role: "readWrite",
            db: "tienda"
        }
    ]
});
```

Ilustración 20. Creación de un usuario administrador para MongoDB (06/11/2026)

Tenemos que activar la necesidad de autenticarse para acceder. Esto lo cambiamos en Docker-compose.yml.

```yaml
services:
  mongo:
    image: mongo:7
    ports: ["27017:27017"]
    command: ["mongod", "--auth"]
    environment: { MONGO_INITDB_DATABASE: tienda }
    volumes:
      - ./init_mongo.js:/docker-entrypoint-initdb.d/init_mongo.js:ro
```

Ilustración 21. Indicación en el YAML de que MongoDB requiere autenticación (06/11/2026)

Mongosh se utiliza como cliente para conectarte a la base de datos, pero Mongod es el que arranca el servidor, y, en este caso, especificamos que es necesario autenticación. Ya que FastAPI se conecta a esta bbdd, hay que darle las credenciales.

```python
app = FastAPI(title="API NoSQL (vulnerable)")
mongo = MongoClient("mongodb://admin:Iamtheadmin@mongo:27017/tienda")
# mongo = MongoClient(os.getenv("MONGO_HOST","mongo"), 27017)
db = mongo[os.getenv("MONGO_DATABASE","tienda")]
```

Ilustración 22. Escritura de las credenciales de MongoDB en la configuración de FastAPI (06/11/2026)

Ocultar Elasticsearch

Eliminamos el puerto del código del Docker-compose para que no sea accesible

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.17.10
  #ports: ["9200:9200"]
  environment:
    discovery.type: single-node
    xpack.security.enabled: "false"
    ES_JAVA_OPTS: "-Xms512m -Xmx512m"
```

Ilustración 23. Comentamos la instrucción del puerto de Elasticsearch para desactivarla (06/11/2026)

Al ocultar Elasticsearch, tenemos que cambiar el init_elastic.sh que habíamos hecho, ya que referencia al localhost:puerto. Podemos ponerlo con Elasticsearch, ya que el contenedor resolverá a la IP del contenedor y lo inicializaremos usando el Docker.

```
#!/bin/sh

curl -X PUT http://elasticsearch:9200/articulos \
-H "Content-Type: application/json" \
-d '{
  "mappings": {
    "properties": {
      "titulo": {"type": "text"},
      "precio": {"type": "integer"}
    }
  }
}'
```

Ilustración 24. Configuración del script de inicialización para apuntar a Elasticsearch (06/11/2026)

Mejorar paginación

Vamos a usar una librería en python que se llama quote_plus de urllib.parse. Nos va a permitir borrar los caracteres especiales de las búsquedas (evitando problemas cuando el string contiene espacios, "?", "=", "#", "&"... porque los elimina o los traduce en su código hexadecimal)

```
from fastapi import FastAPI, Request, HTTPException
from urllib.parse import quote_plus
from pydantic import BaseModel
```

Ilustración 25. Importación de la libreria quote_plus para habilitar la paginación (06/11/2026)

Y la usamos en el get artículos para transformar la cadena que nos llegue.

```
51
52 @app.get("/articulos")
53 def art_search(q: str="", page: int = 1, size: int = 10):
54     q = quote_plus(q)
55
56     return requests.get("http://elasticsearch:9200/articulos/_search",
57                         params={"q": q, "from": (page-1) * size, "size": size}).json()
58
```

Ilustración 26. Uso de la librería quote_plus en la función para obtener artículos (06/11/2026)

Además, añadimos dos atributos a la función, que son página y tamaño, para poder paginar los resultados. De esta manera, los parámetros que vamos a usar son la "q" como búsqueda, el "from" para decidir desde que resultados estamos empezando a ver y el "size" para especificar cuántos artículos mostramos en cada página. Si cogemos los valores predeterminados, es decir, página 1 y size 10, el panel quedaría así: q = "", from = (1-1)*10 = 0, size = 10.

Esto significa que devolvemos el resultado de la búsqueda de q, desde el artículo 0 y como el tamaño es 10, devolveríamos del 1 al 10. Por ejemplo, si tuviese 20 artículos que contuviesen la palabra “lenovo” (q = “lenovo”), los iría devolviendo en páginas, primero del 1 al 10 y luego del 11 al 20. Antes, un usuario usando q = “*” obtenía todos los artículos. Ahora, como está limitado por páginas, solo obtendría los 10 primeros, reduciendo el volumen de datos expuestos en una petición.

TLS

Para poder contar con TLS, necesitamos certificados válidos y en vigor. En este caso, tendríamos que montar una PKI, es decir, infraestructura de clave pública, donde diseñaríamos un certificado x.509 autofirmado para la Autoridad de Certificación que, a su vez, podría expedir certificados para servidores y clientes. En nuestro caso particular, sería muy interesante habilitar los certificados en la conexión entre la FastAPI y MongoDB. De esa forma, los datos viajarían cifrados y se protegerían. Proponemos esa mejora, que se puede llevar a cabo con openssl.

Ratelimit

Para evitar ataques de fuerza bruta, podemos usar un limitador en el login, que bloquee las peticiones a partir de determinados intentos por minuto. Usaremos la biblioteca slowapi.

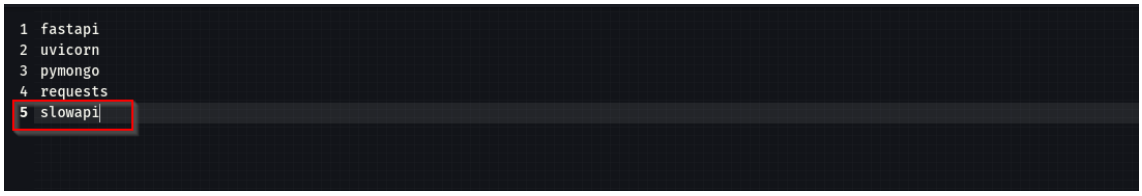


Ilustración 27. Adición de la biblioteca slowapi en requirements.txt (06/11/2026)

La añadimos en requirements.txt y creamos el limitador por IP en el main.py.

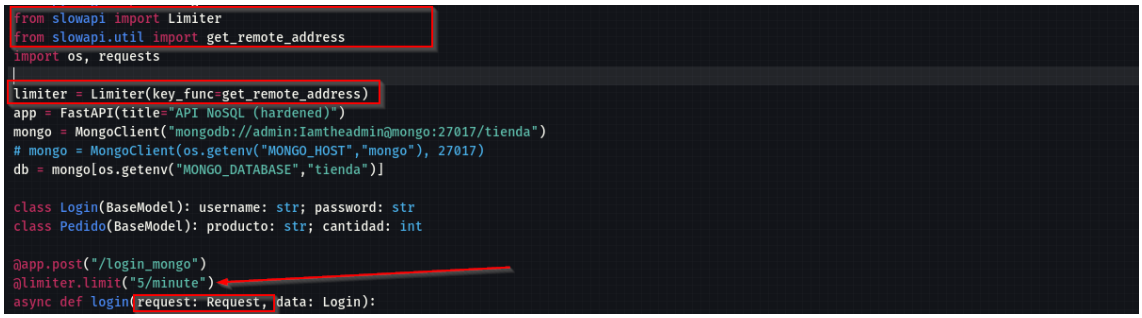


Ilustración 28. Inicialización y uso del limitador en el método login del YAML (06/11/2026)

Intentos de login fallidos

Importamos la biblioteca de logging, la inicializamos y mostramos la alerta en el caso de que no se pueda verificar al usuario.



Ilustración 29. Configuración de alertas de intentos fallidos a través de la biblioteca logging en el YAML (06/11/2026)

Resto de medidas

Backups

Se podrían configurar backups periódicos para garantizar la recuperación ante incidentes. Debemos tener en cuenta que los backups han de comprobarse, es decir, deben restaurarse en una máquina aislada para verificar que funcionan correctamente. Se pueden hacer con comandos como

mongodump --archive=backup.gz –gzip

Picos de búsqueda

En este caso, podríamos tener algún sistema de monitorización, como Kibana o un SIEM, para verificar si existe algún patrón fuera de lo común. Tendríamos que usar el logging que hemos incorporado en nuestro python para añadir la búsqueda realizada en la función de get pedidos.

Verificación de la defensa

Reconstruimos el entorno con todos los cambios que hemos hecho.

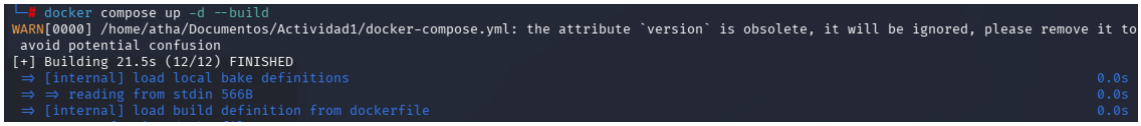


Ilustración 30. Reconstrucción del entorno (06/11/2026)

También ejecutamos el script dentro de Elasticsearch. E intentamos los mismos ataques previos.



Ilustración 31. Ejecutamos el script de inicialización dentro de Elasticsearch (06/11/2026)

NoSQL Injection y rate limit

Podemos intentar tanto el ataque NoSQLi como logarnos con credenciales no válidas. No nos autentica.

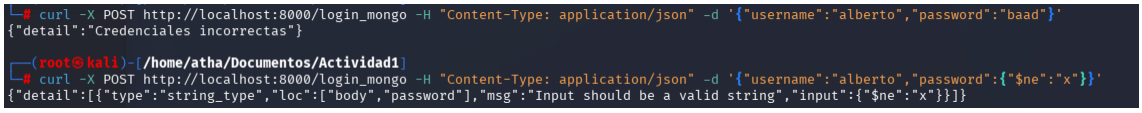


Ilustración 32. Intento de acceso a FastAPI con NoSQLi y credenciales no válidas (06/11/2026)

En los logs de FastAPI vemos cómo se refleja este acceso, con la IP que ha intentado entrar

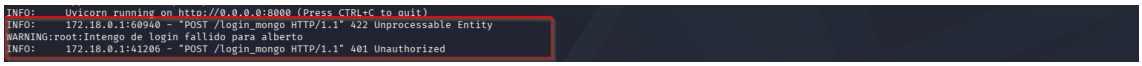


Ilustración 33. Lectura de los errores dentro del log de FastAPI (06/11/2026)

Corregimos la errata en el código “intengo de login”. Vamos a probar ahora varios intentos de login erróneos.



Ilustración 34. Secuencia de intentos de autenticación no válidos hasta el bloqueo (06/11/2026)

Funciona. En el log también aparece.

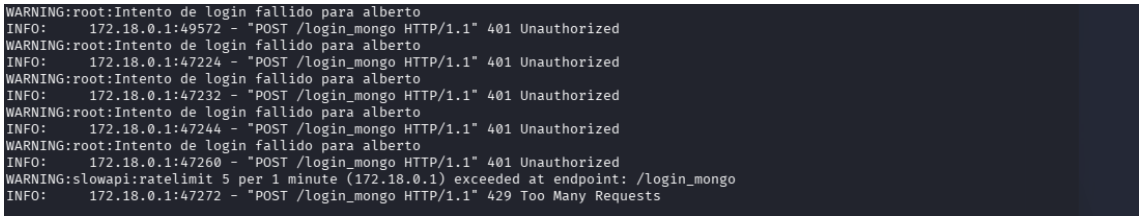


Ilustración 35. Lectura de los intentos fallidos y del bloqueo en el log de FastAPI (06/11/2026)

Autenticación en MongoDB

Si nos conectamos ahora, nos pedirá autenticarnos para cualquier acción de vista o modificación.



Ilustración 36. Error al intentar leer en MongoDB (06/11/2026)

Acceso a Elasticsearch

Al omitir los puertos, no podemos ver ni modificar nada de Elasticsearch



Ilustración 37. Intento fallido de lectura de los índices de Elasticsearch (06/11/2026)

Sin embargo, hay que tener en cuenta que usando la IP del contenedor, aún podemos acceder. Lo que hemos eliminado es el acceso a través del puerto 9200

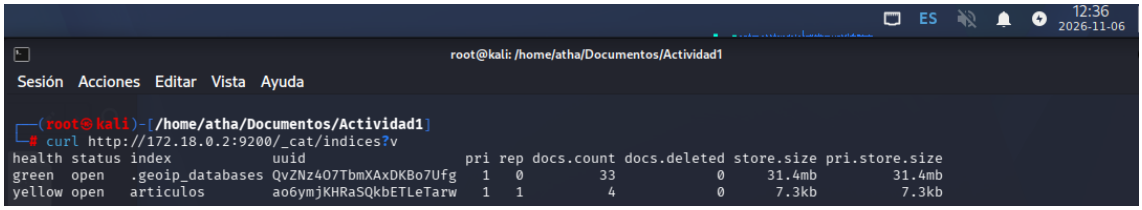


Ilustración 38. Acceso a Elasticsearch a través de su IP

Conclusión

Encontramos diferentes enfoques que se relacionan con este laboratorio. Por una parte, la importancia de realizar un buen hardening en el arranque y la configuración de los sistemas y servicios, lo que reduce su superficie de exposición ante posibles ataques. Esta práctica, ya de por sí, puede resultar muy útil y no tiene por qué interferir con el flujo de trabajo, siempre y cuando se tenga en cuenta desde el principio (lo que en DevSecOps denominarían Shift Left).

Por otra parte, aunque no hayamos profundizado ampliamente en ello, podemos ver algunos ejemplos de aplicación de los principios de mínimo privilegio y Zero Trust. Cuando nos aseguramos de que solo exista un administrador que pueda leer y modificar MongoDB, reducimos la superficie de ataque, permitiendo que únicamente dicho usuario pueda tocar las bases de datos. También reforzamos los controles de autenticación, desconfiando por defecto de cualquier información que puedan introducir los diversos agentes que interactúen con los servicios. Como ejemplo, no podemos asumir que los usuarios vayan a proporcionar siempre datos válidos, por eso se hace imprescindible incorporar mecanismos de validación que descarten entradas potencialmente dañinas o maliciosas.

Si quisiésemos extender estos principios aún más, tendríamos que llevarlos a la conexión existente entre FastAPI y MongoDB, ya que, en este caso, hemos dejado las credenciales de administrador dentro del código main.py de la aplicación, lo cual tampoco sería una práctica recomendable en un entorno real. Lo adecuado sería armar un usuario específico, con los permisos mínimos estrictamente necesarios, y exigir autenticación en cada acceso a la base de datos. Incluso se podrían definir perfiles diferenciados, sirviendo como ejemplo uno con acceso solo de lectura y otro con permisos de escritura solo en determinadas colecciones. Esta implementación funciona como una aplicación más exhaustiva de los principios de Zero Trust y mínimo privilegio.

Concluimos la práctica mostrando la importancia de realizar análisis de vulnerabilidades sobre nuestros servicios (en entornos controlados) para poder identificar y mitigar posibles puntos de compromiso. En ocasiones, la distancia entre un incidente menor y un impacto grave reside en medidas relativamente sencillas: una configuración más segura, una validación adecuada de entradas, una correcta gestión de privilegios o el establecimiento de políticas de seguridad que aborde puntos básicos e imprescindibles, como la rotación de contraseñas, la respuesta a filtraciones, la formación de los usuarios o la realización periódica de simulaciones de phishing.

Bibliografía

Docker. (s.f.). *Install Docker Engine on Debian*. Obtenido de Docker: https://docs.docker.com/engine/install/debian/#install-using-the-repository

Elastic. (s.f.). *Elastic Docs*. Obtenido de Elastic: https://www.elastic.co/docs

Fastapi. (s.f.). *FastAPI en Contenedores - Docker¶*. Obtenido de Fastapi: https://fastapi.tiangolo.com/es/deployment/docker/#recap

HackTricks. (s.f.). *NoSQL injection*. Obtenido de HackTricks: https://hacktricks.wiki/en/pentesting-web/nosql-injection.html

Isaac. (11 de Marzo de 2025). *Guía completa para usar Docker Compose*. Obtenido de Mundo Bytes: https://mundobytes.com/usar-docker-compose/

MongoDB. (s.f.). *Documentación de MongoDB*. Obtenido de MongoDB: https://www.mongodb.com/es/docs/